

# Lab 19 - expected output

Reference transcript for Lab 19 | StatefulSets, Headless Services and Stable Identity, namespace kcna-lab19.

How to read this file. Pod IPs, PV UUIDs, EndpointSlice name suffixes, timestamps and ages are environment-specific and will differ. Pod names, ordinals, DNS names, shard counts, phases and error strings must match exactly.

>

The cluster DNS service IP is 10.96.0.10 on a default kind cluster; yours may differ. Check with `kubectl -n kube-system get svc kube-dns`.

---

## Step 1 - datasets

```
$ for s in shard-0 shard-1 shard-2; do printf '%s = %s entries\n' "$s" "$(grep -c ",$s," data/ledger-entries.csv)"; done
shard-0 = 9 entries
shard-1 = 8 entries
shard-2 = 7 entries
```

These three numbers are load-bearing. Every later check compares Pod output against them.

---

## Step 2 - headless Service

```
$ kubectl -n kcna-lab19 get service depot-ledger
```

| NAME         | TYPE      | CLUSTER-IP | EXTERNAL-IP | PORT(S) | AGE |
|--------------|-----------|------------|-------------|---------|-----|
| depot-ledger | ClusterIP | None       | <none>      | 80/TCP  | 5s  |

Required signal: CLUSTER-IP is the literal string None. TYPE still reads ClusterIP - headless is not a separate Service type, it is a ClusterIP Service with the IP suppressed.

---

## Step 3 - ordered creation

Watch output (second terminal):

|                |     |                 |   |    |
|----------------|-----|-----------------|---|----|
| depot-ledger-0 | 0/1 | Pending         | 0 | 0s |
| depot-ledger-0 | 0/1 | Init:0/1        | 0 | 2s |
| depot-ledger-0 | 0/1 | PodInitializing | 0 | 6s |
| depot-ledger-0 | 1/1 | Running         | 0 | 9s |
| depot-ledger-1 | 0/1 | Pending         | 0 | 0s |
| ...            |     |                 |   |    |
| depot-ledger-2 | 1/1 | Running         | 0 | 8s |

Required signal: ordinal 1 does not appear until ordinal 0 reaches 1/1 Running, and ordinal 2 does not appear until ordinal 1 does. Elapsed times vary with image pull and provisioning speed.

```
$ kubectl -n kcna-lab19 get statefulset,pods
NAME                                READY  AGE
statefulset.apps/depot-ledger      3/3    52s

NAME          READY  STATUS    RESTARTS  AGE
pod/depot-ledger-0  1/1    Running    0          52s
pod/depot-ledger-1  1/1    Running    0          43s
pod/depot-ledger-2  1/1    Running    0          34s
```

Ages descend by roughly the per-Pod startup time - the visible fingerprint of OrderedReady.

---

## Step 4 - one PVC per ordinal

```
$ kubectl -n kcna-lab19 get pvc
NAME                                STATUS  VOLUME                                     CAPACITY  ACCESS
MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
ledger-depot-ledger-0  Bound   pvc-3a71c0e5-9b28-4d16-8f04-c7e2b915d6a3  128Mi     RWO
standard  <unset>  61s
ledger-depot-ledger-1  Bound   pvc-d0c48f92-16ba-4e37-95a1-2f8b60e4c771  128Mi     RWO
standard  <unset>  52s
ledger-depot-ledger-2  Bound   pvc-7e5b1c34-a802-49df-b6c8-51d0937fae28  128Mi     RWO
standard  <unset>  43s
```

Required signal: exactly three claims, named ledger-depot-ledger-<n>, all Bound, each to a \*different\* pvc-<uuid> volume. The UUIDs will differ from these.

```
$ for i in 0 1 2; do printf 'depot-ledger-%s: ' "$i"; kubectl -n kcna-lab19 logs "depot-ledger-$i" -c seed-shard; done
depot-ledger-0: ordinal=0 shard=shard-0 entries=9 mounts=1
depot-ledger-1: ordinal=1 shard=shard-1 entries=8 mounts=1
depot-ledger-2: ordinal=2 shard=shard-2 entries=7 mounts=1
```

Required signal: entries=9, entries=8, entries=7 - matching Step 1 exactly. mounts=1 on a first run.

---

## Step 5 - per-Pod DNS

Headless Service name resolves to the Pod IPs:

```
$ kubectl -n kcna-lab19 exec ledger-client -- nslookup depot-ledger.kcna-lab19.svc.cluster.local
Server: 10.96.0.10
Address: 10.96.0.10:53

Name: depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.52
Name: depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.54
Name: depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.56
```

Required signal: three Address: answers, each a Pod IP from the 10.244.0.0/16 kind Pod CIDR. Ordering is not stable between calls.

Individual member:

```
$ kubectl -n kcna-lab19 exec ledger-client -- nslookup depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
Server: 10.96.0.10
Address: 10.96.0.10:53

Name: depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.54
```

Required signal: exactly one answer.

```
$ kubectl -n kcna-lab19 exec ledger-client -- wget -qO- http://depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local/
<h1>TrackLane depot ledger</h1>
<p>pod: depot-ledger-1</p>
<p>ordinal: 1</p>
<p>depot_code: depot-b</p>
<p>depot_name: Tuas Mega Depot</p>
<p>region: SG-West</p>
<p>ledger_shard: shard-1</p>
<p>shard_entries: 8</p>
<p>volume_mount_count: 1</p>
```

Every field except volume\_mount\_count is fixed by data/depot-roster.csv and data/ledger-entries.csv and must match exactly.

---

## Step 6 - ClusterIP contrast

```
$ kubectl -n kcna-lab19 get svc
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)    AGE
depot-ledger        ClusterIP   None            <none>       80/TCP     6m
depot-ledger-vip    ClusterIP   10.96.184.207   <none>       80/TCP     4s

$ kubectl -n kcna-lab19 exec ledger-client -- nslookup depot-ledger-vip.kcna-lab19.svc.cluster.local
Server: 10.96.0.10
Address: 10.96.0.10:53

Name: depot-ledger-vip.kcna-lab19.svc.cluster.local
Address: 10.96.184.207
```

Required signal: ONE answer, and it is in the Service CIDR (10.96.0.0/12 by default on kind), not the Pod CIDR.

Six requests through the VIP:

```
<p>pod: depot-ledger-2</p>
<p>pod: depot-ledger-0</p>
<p>pod: depot-ledger-2</p>
<p>pod: depot-ledger-1</p>
<p>pod: depot-ledger-0</p>
<p>pod: depot-ledger-1</p>
```

Your sequence will differ and should NOT be a clean rotation. kube-proxy in iptables mode selects a backend probabilistically per connection. Any distribution that touches more than one Pod demonstrates the point.

```
$ kubectl -n kcna-lab19 get endpointslices -o custom-columns='SLICE:.metadata.name,SERVICE:.metadata.labels.kubernetes\.io/service-name,ADDRESSES:.endpoints[*].addresses'
```

| SLICE                  | SERVICE          | ADDRESSES                           |
|------------------------|------------------|-------------------------------------|
| depot-ledger-jm4x9     | depot-ledger     | 10.244.0.52,10.244.0.54,10.244.0.56 |
| depot-ledger-vip-t7q2b | depot-ledger-vip | 10.244.0.52,10.244.0.54,10.244.0.56 |

Identical backends, different DNS behaviour. The five-character slice suffixes are random.

---

## Step 7 - identity survives Pod deletion

Before:

| NAME           | READY | STATUS  | RESTARTS | AGE | IP          | NODE               |
|----------------|-------|---------|----------|-----|-------------|--------------------|
| depot-ledger-1 | 1/1   | Running | 0        | 9m  | 10.244.0.54 | kcna-control-plane |

After delete and recreate:

| NAME           | READY | STATUS  | RESTARTS | AGE | IP          | NODE               |
|----------------|-------|---------|----------|-----|-------------|--------------------|
| depot-ledger-1 | 1/1   | Running | 0        | 14s | 10.244.0.61 | kcna-control-plane |

Required signal: the name is identical, the IP has changed. If the IP happens to be reused, the test still holds - the mount counter below is the decisive evidence.

```
$ kubectl -n kcna-lab19 logs depot-ledger-1 -c seed-shard
ordinal=1 shard=shard-1 entries=8 mounts=2

$ kubectl -n kcna-lab19 exec depot-ledger-1 -c web -- sh -c 'cat /usr/share/nginx/html/mounts.log'
```

|                        |                      |
|------------------------|----------------------|
| mounted depot-ledger-1 | 2026-09-05T06:31:12Z |
| mounted depot-ledger-1 | 2026-09-05T06:40:47Z |

Required signal: mounts=2 and two lines in mounts.log with different timestamps. mounts=1 would mean a fresh volume was provisioned, i.e. the identity guarantee failed.

---

## Step 8 - reverse-ordered scale-down, retained claims

|                |     |             |   |     |
|----------------|-----|-------------|---|-----|
| depot-ledger-2 | 1/1 | Terminating | 0 | 14m |
| depot-ledger-2 | 0/1 | Completed   | 0 | 14m |
| depot-ledger-1 | 1/1 | Terminating | 0 | 4m  |
| depot-ledger-1 | 0/1 | Completed   | 0 | 4m  |

Required signal: ordinal 2 terminates before 1. Ordinal 0 is untouched.

```
$ kubectl -n kcna-lab19 get pvc
```

|                       |       |     |       |     |          |         |     |
|-----------------------|-------|-----|-------|-----|----------|---------|-----|
| ledger-depot-ledger-0 | Bound | ... | 128Mi | RWO | standard | <unset> | 15m |
| ledger-depot-ledger-1 | Bound | ... | 128Mi | RWO | standard | <unset> | 15m |
| ledger-depot-ledger-2 | Bound | ... | 128Mi | RWO | standard | <unset> | 15m |

Required signal: all three claims survive with only one Pod running. This is whenScaled: Retain.

After scaling back to 3:

```
$ kubectl -n kcna-lab19 logs depot-ledger-2 -c seed-shard
ordinal=2 shard=shard-2 entries=7 mounts=2
```

mounts=2 proves ordinal 2 re-adopted its pre-scale-down volume.

---

## Step 9 - Parallel pod management

```
$ kubectl -n kcna-lab19 get pods -l app.kubernetes.io/name=lane-indexer
```

| NAME           | READY | STATUS            | RESTARTS | AGE |
|----------------|-------|-------------------|----------|-----|
| lane-indexer-0 | 0/1   | ContainerCreating | 0        | 2s  |
| lane-indexer-1 | 0/1   | ContainerCreating | 0        | 2s  |
| lane-indexer-2 | 0/1   | ContainerCreating | 0        | 2s  |

Required signal: all three at the same age, created concurrently. Contrast with Step 3's staggered ages.

```
$ kubectl -n kcna-lab19 exec ledger-client -- wget -qO- http://lane-indexer-2.lane-indexer.
kcna-lab19.svc.cluster.local/
lane-indexer ordinal 2 ready

$ kubectl -n kcna-lab19 get pvc -l app.kubernetes.io/name=lane-indexer
No resources found in kcna-lab19 namespace.
```

Stable DNS identity with no per-Pod storage - proving the two features are independent.

---

## Graded checks

```
$ bash verification/checks.sh
== Lab 19 verification - namespace kcna-lab19 ==
[PASS] namespace kcna-lab19 exists
[PASS] service/depot-ledger is headless (clusterIP: None)
[PASS] service/depot-ledger-vip has an allocated ClusterIP
[PASS] statefulset/depot-ledger has serviceName=depot-ledger
[PASS] statefulset/depot-ledger reports 3/3 ready replicas
[PASS] pods are named by ordinal: depot-ledger-0, -1, -2
[PASS] one PVC exists per ordinal from volumeClaimTemplates
[PASS] each pod mounts its own ordinal's PVC
[PASS] shard entry counts 9/8/7 match data/ledger-entries.csv
[PASS] per-pod DNS resolves depot-ledger-1.depot-ledger.kcna-lab19.svc.cluster.local
[PASS] headless DNS returns 3 pod addresses, VIP DNS returns 1
[PASS] depot-ledger-1 reattached its original volume (mount count 3)
[PASS] persistentVolumeClaimRetentionPolicy is Retain/Retain
[PASS] statefulset/lane-indexer uses podManagementPolicy=Parallel
[PASS] no container in kcna-lab19 requests privileged: true
-----
15 passed, 0 failed
```

Exit code 0. The reported mount count is 2 if you ran the checks straight after Step 7, or 3 after the Step 8 scale cycle - both pass.

---

## Failure injection - missing headless Service

The workload looks perfectly healthy:

```
$ kubectl -n kcna-lab19 get statefulset ghost-ledger
NAME          READY   AGE
ghost-ledger  1/1     31s

$ kubectl -n kcna-lab19 get pod ghost-ledger-0
NAME          READY   STATUS    RESTARTS   AGE
ghost-ledger-0  1/1     Running   0           31s
```

There are no Warning events on either object. That is the trap.

```
$ kubectl -n kcna-lab19 exec ledger-client -- nslookup ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local
Server: 10.96.0.10
Address: 10.96.0.10:53

** server can't find ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local: NXDOMAIN

command terminated with exit code 1
```

Required signal: the literal token NXDOMAIN and a non-zero exit code.

```
$ kubectl -n kcna-lab19 exec ledger-client -- wget -T 5 -q0- http://ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local/
wget: bad address 'ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local'
command terminated with exit code 1
```

bad address (not connection refused, not a timeout) confirms name resolution, not connectivity, is the fault.

```
$ kubectl -n kcna-lab19 get service ghost-ledger
Error from server (NotFound): services "ghost-ledger" not found

$ kubectl -n kcna-lab19 get endpointslices -l kubernetes.io/service-name=ghost-ledger
No resources found in kcna-lab19 namespace.
```

Both confirm the root cause. After creating the headless Service:

```
$ kubectl -n kcna-lab19 exec ledger-client -- nslookup ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local | tail -2
Name: ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local
Address: 10.244.0.71
```

DNS propagation through CoreDNS takes a few seconds; retry if the first attempt still returns NXDOMAIN.

---

## Cleanup

```
$ kubectl -n kcna-lab19 delete statefulset depot-ledger lane-indexer --ignore-not-found
statefulset.apps "depot-ledger" deleted
statefulset.apps "lane-indexer" deleted

$ kubectl -n kcna-lab19 get pvc
NAME                                STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS     
VOLUMEATTRIBUTESCLASS             AGE
ledger-depot-ledger-0              Bound    ...      128Mi      RWO             standard        <unset>
44m
ledger-depot-ledger-1              Bound    ...      128Mi      RWO             standard        <unset>
44m
ledger-depot-ledger-2              Bound    ...      128Mi      RWO             standard        <unset>
44m
```

Required signal: the claims outlive the StatefulSet. This is the lesson, not a bug.

```
$ kubectl -n kcna-lab19 delete pvc ledger-depot-ledger-0 ledger-depot-ledger-1 ledger-depot-
ledger-2
persistentvolumeclaim "ledger-depot-ledger-0" deleted
persistentvolumeclaim "ledger-depot-ledger-1" deleted
persistentvolumeclaim "ledger-depot-ledger-2" deleted

$ kubectl get pv | grep kcna-lab19 || echo "all dynamically provisioned volumes reclaimed"
all dynamically provisioned volumes reclaimed

$ kubectl delete namespace kcna-lab19 --wait=true
namespace "kcna-lab19" deleted

$ kubectl get namespace kcna-lab19
Error from server (NotFound): namespaces "kcna-lab19" not found
```

Required end state: no kcna-lab19 namespace, no leftover PVCs and no leftover PVs. This lab creates no cluster-scoped objects.